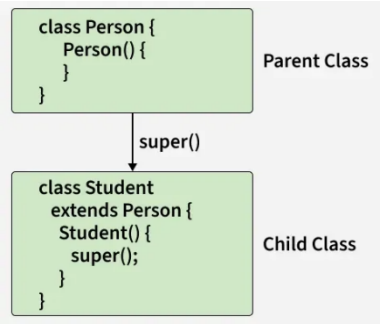
super Keyword in Java

The super keyword in Java is used to refer to the immediate parent class object. It is commonly used to access parent class methods and constructors, enabling a subclass to inherit and reuse the functionality of its superclass.



Usage

The super keyword can be used in three primary contexts:

1. To call the superclass constructor.
2. To access a method from the superclass that has been overridden in the subclass.
3. To access a field from the superclass when it is hidden by a field of the same name in the subclass.

Syntax

```
super();

super.methodName();

super.fieldName;
```

Examples

Example 1: Calling Superclass Constructor

```
class Animal {
   Animal() {
      System.out.println("Animal constructor called");
   }
}
```

```
class Dog extends Animal {
    Dog() {
        super(); // Calls the constructor of Animal class
        System.out.println("Dog constructor called");
    }
}

public class TestSuper {
    public static void main(String[] args) {
        Dog d = new Dog(); // Output: Animal constructor called
                           //         Dog constructor called
    }
}
```

In this example, the super() call in the Dog constructor invokes the constructor of the Animal class.

Example 2: Accessing Superclass Method

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void sound() {
        super.sound(); // Calls the sound() method of Animal class
        System.out.println("Dog barks");
```

```
    }
}


public class TestSuper {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound(); // Output: Animal makes a sound
                //        Dog barks
    }
}
```

Here, the super.sound() call in the Dog class method sound() invokes the sound() method of the Animal class.

Example 3: Accessing Superclass Field

```
class Animal {
    String color = "white";
}


class Dog extends Animal {
    String color = "black";


    void displayColor() {
        System.out.println("Dog color: " +// Prints the color of Dog class
        System.out.println("Animal color: " + super.// Prints the color of Animal class
    }
}
```

```
public class TestSuper {
   public static void main(String[] args) {
      Dog d = new Dog();
      d.displayColor(); // Output: Dog color: black
                        //         Animal color: white
   }
}
```

In this example, super.color is used to access the color field of the Animal class, which is hidden by the color field in the Dog class.

Tips and Best Practices

Constructor Chaining: Use super() in the constructor of a subclass to ensure that the constructor of the superclass is called, enabling proper initialization of the object.

```
class Parent {
   Parent() {
      System.out.println("Parent constructor");
   }
}

class Child extends Parent {
   Child() {
      super(); // Always the first statement in the constructor
      System.out.println("Child constructor");
   }
}
```

Method Overriding: Use super.methodName() to call a method from the superclass when it is overridden in the subclass. This is useful for extending or modifying the behavior of the inherited method.

```
class Parent {
    void display() {
        System.out.println("Parent display");
    }
}

class Child extends Parent {
    void display() {
        super.display(); // Calls Parent's display method
        System.out.println("Child display");
    }
}
```

Field Access: Use super.fieldName to access a field from the superclass when it is hidden by a field of the same name in the subclass. This helps to avoid ambiguity and ensures that the correct field is accessed.

```
class Parent {
    int number = 10;
}

class Child extends Parent {
    int number = 20;

    void showNumber() {
        System.out.println(super.number); // Accesses Parent's number
```

```
    }
}
```

Readability: Use super judiciously to make your code more readable and maintainable. Overusing super can make the code harder to understand.

Placement in Constructors: Always place the super() call as the first statement in the subclass constructor to avoid compilation errors.

Static Contexts: Remember that super cannot be used in static methods or static contexts, as it refers to an instance of a class.

For more examples visit:

https://www.geeksforgeeks.org/java/super-keyword/